

Computer Architecture

Microarchitecture

- Implementation of the ISA under specific **design constraints and goals** أى شئ يحدث فى الهاردوير دون التعرض للسوفت وير
- Anything done in hardware without exposure to software
 - Pipelining
 - In-order versus out-of-order instruction execution
 - Memory access scheduling policy
 - **Superscalar** processing (**multiple instruction issue?**)
 - Clock gating
 - Caching? Levels, size, associativity, replacement policy
 - Prefetching?
 - Voltage/frequency scaling?
 - Error correction?

Example

- Out of order (memory **reordering**)

In a modern (**super scalar**) processor that can execute **2** or more instruction at once. **Single core** processor not multi-core

1) mov eax,0

2) mov edx,1

Solution1: (1,2) → (3) → (4,5)

3) **move edx,3**

Solution2: (1,2) → (**3,5**) → (4,...)

4) **inc edx**

5) mov ecx,3

- The **transistors** only **decide** which solution to perform

Property of ISA vs. Uarch?

- ADD instruction's opcode Arch
 - Number of general purpose registers Arch
 - Number of ports to the register file UArch
 - Number of cycles to execute the MUL instruction UArch
 - Whether or not the machine employs pipelined instruction execution UArch
-
- Remember
 - Microarchitecture: Implementation of the ISA under specific design constraints and goals

Computer Architecture

ISA Tradeoffs

Application Space

■ Dream, and they will appear...

Other examples of the application space that continue to drive the need for unique design points are the following:

- 1) scientific applications such as those whose computations control nuclear power plants, determine where to drill for oil, and predict the weather;
- 2) transaction-based applications such as those that handle ATM transfers and e-commerce business;
- 3) business data processing applications, such as those that handle inventory control, payrolls, IRS activity, and various personnel record keeping, whether the personnel are employees, students, or voters;
- 4) network applications, such as high-speed routing of Internet packets, that enable the connection of your home system to take advantage of the Internet;
- 5) guaranteed delivery (a.k.a. real time) applications that require the result of a computation by a certain critical deadline;
- 6) embedded applications, where the processor is a component of a larger system that is used to solve the (usually) dedicated application;
- 7) media applications such as those that decode video and audio files;
- 8) random software packages that desktop users would like to run on their PCs.

Each of these application areas has a very different set of characteristics. Each application area demands a different set of tradeoffs to be made in specifying the microprocessor to do the job.

Design Point

- A set of design considerations and their importance

- **leads to tradeoffs** in both ISA and uarch

- Considerations

- 1 □ Cost
- 2 □ Performance
- 3 □ Maximum power consumption
- 4 □ Energy consumption (battery life)
- 5 □ Availability
- 6 □ Reliability and Correctness
- 7 □ Time to Market

Problem
Algorithm
Program
ISA
Microarchitecture
Circuits
Electrons

Design point determined by the “Problem” space (application space), the intended users/*market*

Many Different ISAs Over Decades

- x86
- PDP-x: Programmed Data Processor (PDP-11)
- VAX
- IBM 360
- CDC 6600
- SIMD ISAs: CRAY-1, Connection Machine
- VLIW ISAs: Multiflow, Cydrome, IA-64 (EPIC)
- PowerPC, POWER
- RISC ISAs: Alpha, MIPS, SPARC, ARM
- What are the fundamental differences?
 - E.g., how instructions are specified and what they do
 - E.g., how complex are the instructions

Instruction

- Basic element of the HW/SW interface

- Consists of

- **opcode**: what the instruction does
- **operands**: who it is to do it to

- Example from the **Alpha** ISA:

RISC

31	26	25	21	20	16	15	5	4	0	
Opcode	Number									PALcode Format
Opcode	RA	Disp								Branch Format
Opcode	RA	RB	Disp							Memory Format
Opcode	RA	RB	Function					RC	Operate Format	

push 5

if , for , if else

3 3 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0

Cond	0	0	1	Opcode				S	Rn		Rd		Operand 2								<i>Data Processing / PSR Transfer</i>			
Cond	0	0	0	0	0	0	A	S	Rd		Rn		Rs		1	0	0	1	Rm		<i>Multiply</i>			
Cond	0	0	0	0	1	U	A	S	RdHi		RdLo		Rn		1	0	0	1	Rm		<i>Multiply Long</i>			
Cond	0	0	0	1	0	B	0	0	Rn		Rd		0	0	0	0	1	0	0	1	Rm		<i>Single Data Swap</i>	
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn		<i>Branch and Exchange</i>
Cond	0	0	0	P	U	0	W	L	Rn		Rd		0	0	0	0	1	S	H	1	Rm		<i>Halfword Data Transfer: register offset</i>	
Cond	0	0	0	P	U	1	W	L	Rn		Rd		Offset				1	S	H	1	Offset		<i>Halfword Data Transfer: immediate offset</i>	
Cond	0	1	1	P	U	B	W	L	Rn		Rd		Offset								<i>Single Data Transfer</i>			
Cond	0	1	1													1						<i>Undefined</i>		
Cond	1	0	0	P	U	S	W	L	Rn		Register List										<i>Block Data Transfer</i>			
Cond	1	0	1	L	Offset																<i>Branch</i>			
Cond	1	1	0	P	U	N	W	L	Rn		CRd		CP#		Offset						<i>Coprocessor Data Transfer</i>			
Cond	1	1	1	0	CP Opc			CRn		CRd		CP#		CP		0	CRm			<i>Coprocessor Data Operation</i>				
Cond	1	1	1	0	CP Opc			L	CRn		Rd		CP#		CP		1	CRm			<i>Coprocessor Register Transfer</i>			
Cond	1	1	1	1	Ignored by processor																<i>Software Interrupt</i>			

3 3 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 9 8 7 6 5 4 3 2 1 0
1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0

Figure 4-1: ARM instruction set formats

Set of Instructions, Encoding, and Spec

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			A	op.spec				
AND ⁺	0101				DR			SR1			A	op.spec				
BR	0000				n	z	p	PCOffset9								
JMP	1100				000			BaseR			000000					
JSR(R)	0100				A	operand.specifier										
LDB ⁺	0010				DR			BaseR			boffset6					
LDW ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCOffset9								
RTI	1000				000000000000											
SHF ⁺	1101				DR			SR			A	D	amount4			
STB	0011				SR			BaseR			boffset6					
STW	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
XOR ⁺	1001				DR			SR1			A	op.spec				
not used	1010															
not used	1011															

- x86 Manual
- Why unused instructions?
- Aside: concept of “bit steering”
 - A bit in the instruction determines the interpretation of other bits

What Are the Elements of An ISA?

■ Instruction sequencing model

- Control flow vs. data flow
- Tradeoffs?

■ Instruction processing style

- Specifies the number of “operands” an instruction “operates” on and how it does so
- 0, 1, 2, 3 address machines
 - **0**-address: **stack machine** (op, push A, pop A) NO OPERANDS
 - 1-address: **accumulator machine** (op ACC, ld A, st A)
 - 2-address: 2-operand machine (op S,D; one is both source and dest)
 - 3-address: 3-operand machine (op S1,S2,D; source and dest separate)
- Tradeoffs? See your homework question
 - Larger operate instructions vs. more executed operations
 - Code size vs. execution time vs. on-chip memory space

An Example: Stack Machine

- + Small instruction size (no operands needed for operate instructions)
 - ❑ Simpler logic
 - ❑ Compact code
- + Efficient procedure calls: all parameters on stack
 - ❑ **No additional cycles** for parameter passing
- Computations that are not easily expressible with “postfix notation” are difficult to map to **stack machines**
 - ❑ **Cannot** perform operations on **many values at the same time** (only top N values on the stack at the same time)
 - ❑ Not flexible

An Example: Stack Machine Operation

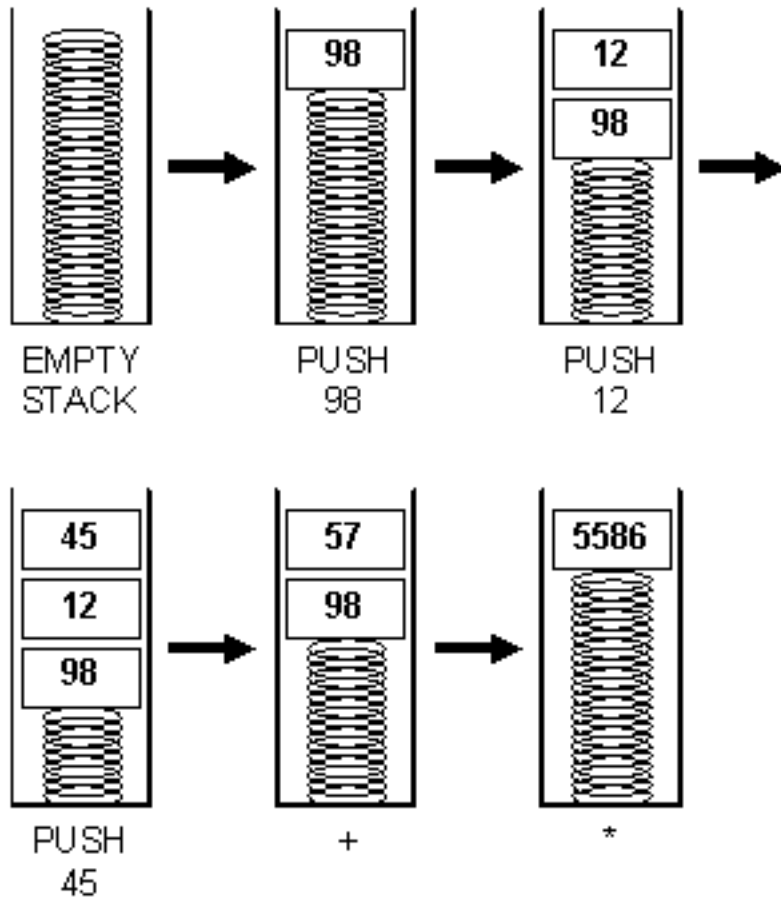


Figure 3.2 -- An example stack machine.

Other Examples

- PDP-11: A **2-address machine**

- PDP-11 ADD: 4-bit opcode, 2 6-bit operand specifiers
- Why? Limited bits to specify an instruction
- Disadvantage: **One source operand** is always clobbered with the **result of the instruction**
 - *How do you ensure you preserve the **old value of the source**?*

- X86: A **2-address** (memory/memory) machine

- Alpha: A **3-address** (load/store) machine

- MIPS?

- ARM?

What Are the Elements of An ISA?

■ Data types

- ❑ Definition: Representation of information for which there are instructions that operate on the representation
- ❑ Integer, floating point, character, binary, decimal, BCD
- ❑ Doubly linked list, **queue**, string, bit vector, **stack**
 - **VAX**: INSQUEUE and REMQUEUE instructions on a doubly linked list or queue; FINDFIRST
 - X86: SCAN opcode operates on character strings; PUSH/POP

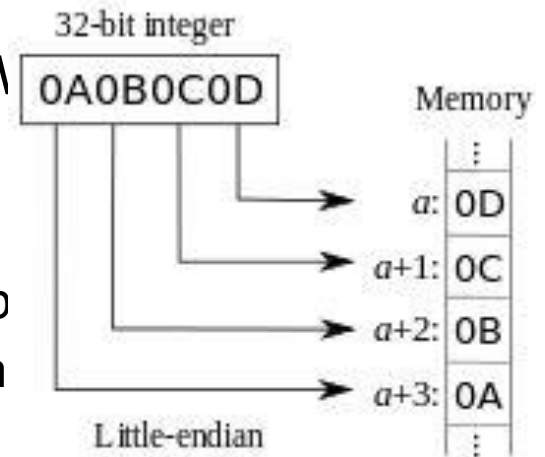
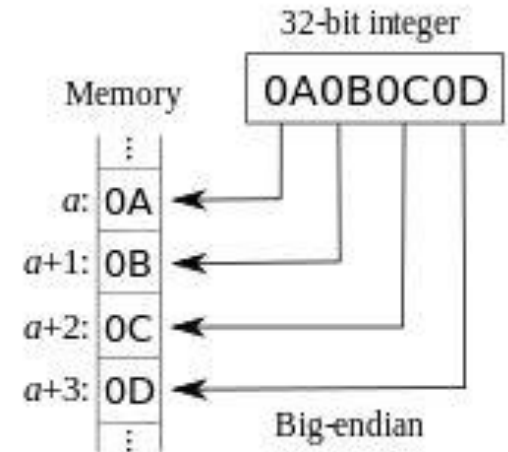
Data Type Tradeoffs

- What is the benefit of having more or high-level data types in the ISA?
- What is the disadvantage?
- Think compiler/programmer vs. microarchitecture
- Concept of **semantic gap**
 - Data types coupled tightly to the semantic level, or **complexity** of instructions
- Example: Early RISC architectures vs. Intel 432
 - Early **RISC**: Only **integer** data type
 - Intel 432: Object data type, capability based machine

What Are the Elements of An ISA?

■ Memory organization

- ❑ Address space: How many uniquely identifiable memory locations
- ❑ Addressability: How much data does each memory location store
 - Byte addressable: most ISAs, characters are
 - Bit addressable: Burroughs 1700. Why?
 - 64-bit addressable: Some supercomputers.
 - 32-bit addressable: First Alpha
 - Food for thought
 - ❑ How do you add 2 32-bit numbers with only byte addressability?
 - ❑ How do you **Big endian vs. little endian?** with only 32-bit addressability?
 - ❑ MSB at low or high byte.
- ❑ Support for virtual memory



What Are the Elements of An ISA?

■ Big-Endian

- makes sense if you view your memory as starting at the **top-left** and addresses **increasing as you go down**
- PPC, Sparc
- **MS** bytes put at the starting address

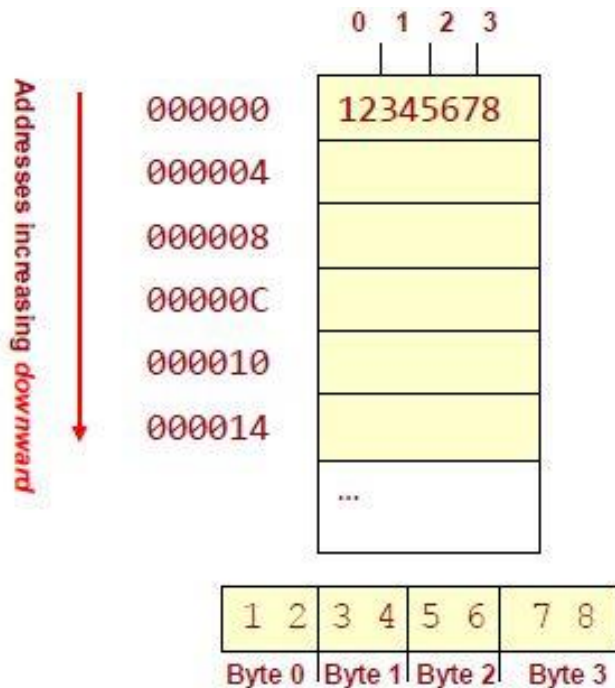
■ Little-Endian

- makes sense if you view your memory as starting at the **bottom-right** and addresses **increasing as you go up**
- used by **Intel processors** / original PCI bus
- **LS** bytes put at the starting address

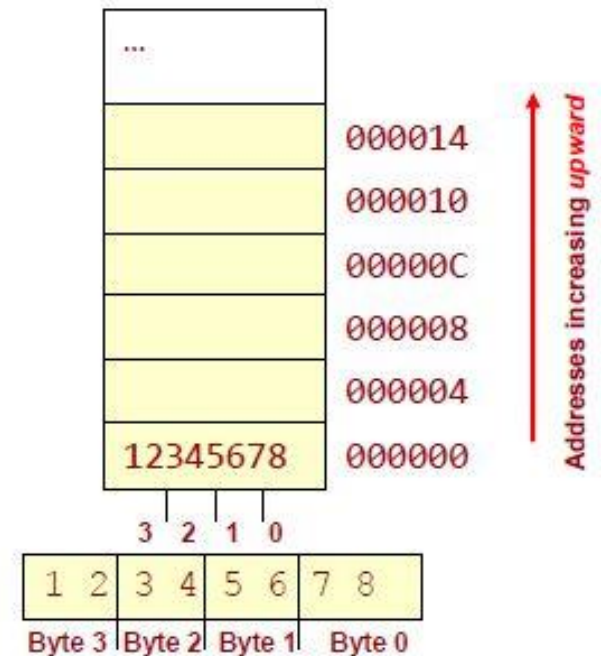
What Are the Elements of An ISA?

■ Memory organization

Big-Endian



Little-Endian



What Are the Elements of An ISA?

■ Registers

- How many
- Size of each register

■ Why is having registers a good idea?

- Because programs exhibit a characteristic called **data locality**
- A recently produced/accessed value is likely to be used more than once (temporal locality)
 - Storing that value in a **register** **eliminates** the need to go to memory each time that value is needed

Aside: Programmer Invisible State

- Microarchitectural state
- Programmer **cannot access** this directly
- E.g. cache state
- E.g. **pipeline registers**

Evolution of Register Architecture

- Accumulator
 - a legacy from the “adding” machine days
- Accumulator + address registers
 - need register indirection
 - initially address registers were special-purpose, i.e., can only be loaded with an address for indirection
 - eventually arithmetic on addresses became supported
- General purpose registers (GPR)
 - all registers good for all purposes
 - grew from a few registers to 32 (common for RISC) to 128 in Intel IA-64

Instruction Classes

- Operate instructions
 - ❑ **Process data:** arithmetic and logical operations
 - ❑ **Fetch** operands, **compute** result, **store** result
 - ❑ Implicit sequential control flow
- Data movement instructions
 - ❑ Move data between memory, registers, I/O devices
 - ❑ Implicit sequential control flow
- Control flow instructions
 - ❑ Change the sequence of instructions that are executed

What Are the Elements of An ISA?

Addressing Mode

- Load/store vs. memory/memory architectures
 - **Load/store** architecture: operate instructions operate **only** on **registers**
 - E.g., MIPS, **ARM** and many **RISC ISAs**
 - **Memory** architecture: operate instructions can operate on **memory** locations
 - E.g., **x86**, VAX and many **CISC ISAs**

RISC - smaller, simpler instruction set. faster clock speeds. more general purpose registers.

CISC - larger, more feature-rich instruction set (more operations, addressing modes, etc.). slower clock speeds. fewer general purpose registers.

ISA-level Tradeoffs: Addressing Modes

- Addressing mode specifies how to obtain an operand of an instruction
 - Register
 - Immediate
 - Memory (displacement, register indirect, indexed, absolute, memory indirect, autoincrement, autodecrement, ...)
- More modes:
 - + help better support **programming** constructs (arrays, pointer-based accesses)
 - make it harder for the **architect to design**
 - too many choices for the compiler?
 - Many ways to do the same thing complicates compiler design
 - *Wulf, “[Compilers and Computer Architecture](#),” IEEE Computer 1981*

What Are the Benefits of Different Addressing Modes?

- Another example of programmer vs. microarchitect tradeoff
- **Advantage** of more addressing modes:
 - Enables **better mapping of high-level constructs to the machine**: some accesses are better expressed with a different mode → reduced number of instructions and code size
 - Think **array accesses** (autoincrement mode)
 - Think indirection (pointer chasing)
 - Sparse matrix accesses
- **Disadvantage**:
 - More work for the **compiler**
 - More work for the **microarchitect**

ISA Orthogonality

- Orthogonal ISA:

is an **instruction set architecture** where all instruction types can use **all addressing modes**

- Example: VAX

- Who is this good for?

- Who is this bad for?

ISA-level Tradeoffs: Instruction Length

- **Fixed length:** Length of all instructions the same
 - + Easier to decode single instruction in hardware
 - + Easier to decode multiple instructions concurrently
 - Wasted bits in instructions (Why is this bad?)
 - Harder-to-extend ISA (how to add new instructions?)
- **Variable length:** Length of instructions different (determined by opcode and sub-opcode)
 - + Compact encoding (Why is this good?)
 - Intel 432: Huffman encoding (sort of). 6 to 321 bit instructions. How?
 - **More logic** to decode a single instruction
 - Harder to decode multiple instructions concurrently
- **Tradeoffs**
 - Code size (memory space, bandwidth, latency) vs. hardware complexity
 - ISA extensibility and expressiveness vs. hardware complexity
 - Performance? Smaller code vs. ease of decode

ISA-level Tradeoffs: Uniform Decode

- **Uniform decode:** Same bits in each instruction correspond to the same meaning
 - **Opcode** is always in the same location
 - operand specifiers, immediate values, ...
 - Many “RISC” ISAs: Alpha, MIPS, SPARC
 - + **Easier** decode, simpler hardware
 - + Enables parallelism: generate target address before knowing the instruction is a branch
 - Restricts instruction format (fewer instructions?) or **wastes space**

- **Non-uniform decode**
 - E.g., opcode can be the 1st-7th byte in x86
 - + More compact and **powerful** instruction format
 - More **complex decode logic**

A Note on Length and Uniformity

- **Uniform decode** usually goes with **fixed length**
- In a variable length ISA, uniform decode can be a property of instructions of the same length
It is hard to think of it as a property of instructions of different lengths

Complex vs. Simple Instructions

- Complex instruction: An instruction does a lot of work, e.g. many operations
 - Insert in a doubly linked list
 - String copy
- Simple(Reduced) instruction: An instruction does small amount of work, it is a primitive using which complex operations can be built
 - Add
 - XOR
 - Multiply

Complex vs. Simple Instructions

- Advantages of Complex instructions
 - + Denser encoding → smaller code size → better memory utilization, saves off-chip bandwidth, better cache hit rate (better packing of instructions)
 - + Simpler compiler: no need to optimize small instructions as much

- Disadvantages of Complex Instructions
 - Larger chunks of work → compiler has less opportunity to optimize (limited in fine-grained optimizations it can do)
 - More complex hardware → translation from a high level to control signals and optimization needs to be done by hardware

ISA-level Tradeoffs: Semantic Gap

- Where to place the ISA? Semantic gap
 - Closer to high-level language (HLL) → **Small semantic gap, complex instructions**
 - Closer to hardware control signals? → Large semantic gap, simple instructions
- RISC vs. CISC machines
 - RISC: Reduced instruction set computer
 - CISC: Complex instruction set computer
 - FFT, QUICKSORT, POLY, FP instructions?
 - VAX INDEX instruction (array access with bounds checking)

A Note on RISC vs. CISC

■ Usually, ... أنا حزين

■ RISC

- ❑ Simple instructions
- ❑ Fixed length
- ❑ Uniform decode
- ❑ Few addressing modes

faster clock speeds
more general purpose registers

■ CISC

- ❑ Complex instructions
- ❑ Variable length
- ❑ Non-uniform decode
- ❑ Many addressing modes

slower clock speeds
fewer general purpose registers

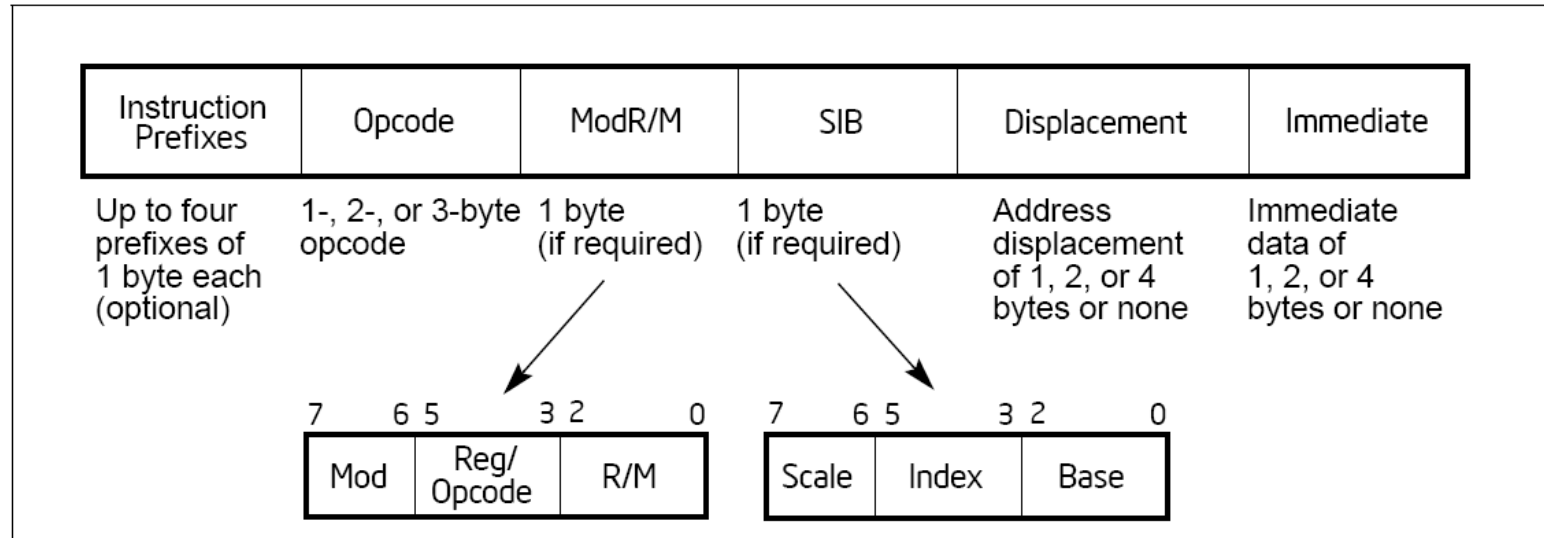
A Note on ISA Evolution

- ISAs have evolved to reflect/satisfy the concerns of the day
- Examples:
 - ❑ Limited on-chip and off-chip memory size
 - ❑ Limited compiler optimization technology
 - ❑ Limited memory bandwidth
 - ❑ Need for specialization in important applications (e.g., MMX)
- Use of translation (in HW and SW) enabled underlying implementations to be similar, regardless of the ISA
 - ❑ Concept of dynamic/static **interface**: translation/interpretation
 - ❑ Contrast it with hardware/software interface

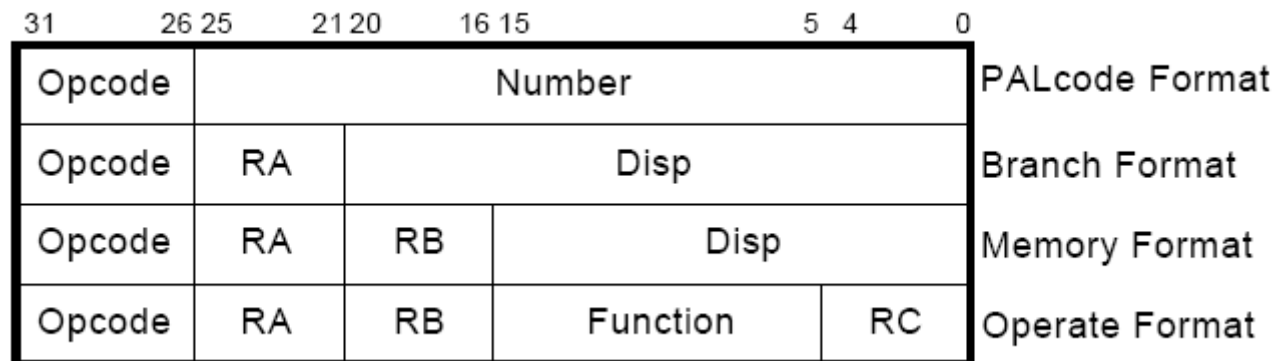
مش
فاهمك
ياض

x86 vs. Alpha Instruction Formats

■ x86:

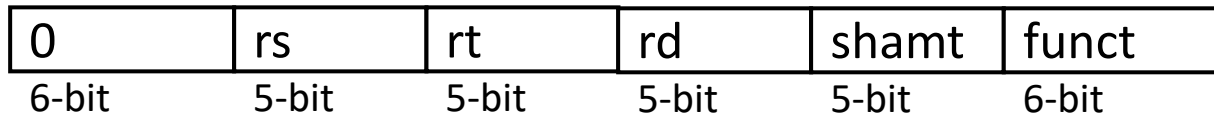


■ Alpha:



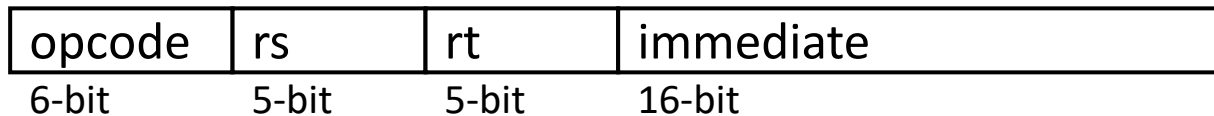
MIPS Instruction Format

- R-type, 3 register operands



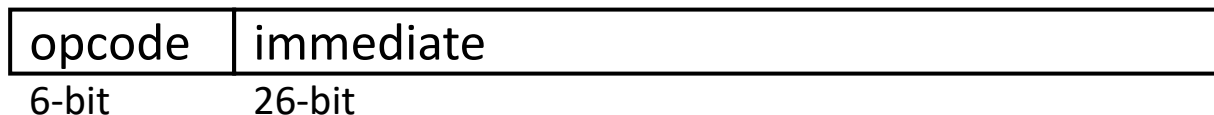
R-type

- I-type, 2 register operands and 16-bit immediate operand



I-type

- J-type, 26-bit immediate operand



J-type

- Simple Decoding

- ❑ 4 bytes per instruction, regardless of format
- ❑ must be 4-byte aligned (2 lsb of PC must be 2b'00)
- ❑ format and fields easy to extract in hardware